J Vis (2024) 27:925–940
https://doi.org/10.1007/s12650-024-00991-1

REGULAR PAPER

Shuhang Zhang · Ruihong Xu · Qing Zhang · Yining Quan · Qi Liu

DeepFD: a deep learning approach to fast generate force-directed layout for large graphs

Received: 22 December 2023 / Revised: 22 December 2023 / Accepted: 20 February 2024 / Published online: 31 May 2024
© The Visualization Society of Japan 2024

Abstract Deep learning techniques have been applied to the graph drawing of node-link diagrams to help figure out user preference of layout in recent research. However, when revisiting existing studies, only stress model and dimensional reduction methods are utilized in the unsupervised learning of graph drawing tasks since their gradient descent conditions can be easily constructed, and few works have explored their scalability on large graphs. In this paper, we propose a framework that can adapt most of the graph layout methods to a form of loss function and develop an implementation DeepFD, which takes the force-directed algorithm as the prototype to design the loss function. Our model is built with the graph-LSTM as encoder and multilayer perceptron as decoder and trained with dataset split from huge graphs with millions of nodes by Louvain. We design a set of qualitative and quantitative experiments to evaluate our method and compare with classical layout techniques such as F-R and K-K algorithms, while deep-learning based models with various architecture or loss function are adopted to perform ablation experiments. The results indicate that our developed approach can generate a high-quality layout of large graph within a low time cost, and the model we propose shows strong robustness and high efficiency.

Keywords Graph layout · Deep learning · Graph-LSTM · Loss function

S. Zhang · R. Xu · Q. Zhang · Y. Quan (✉)
School of Computer Science and Technology, Xidian University, Xi'An, China
E-mail: ynquan@xidian.edu.cn

S. Zhang
E-mail: 847122272@qq.com

R. Xu
E-mail: 1027081646@qq.com

Q. Zhang
E-mail: 1471690846@qq.com

Q. Liu
China Telecom Bestpay Co., Ltd., Shenzhen, China
E-mail: liuqihj@chinatelecom.cn

Present Address:
S. Zhang
Data Analysis Application and Security Assess Lab, Shanghai Motor Vehicle Inspection Certification & Tech Innovation Center Co., Ltd., Shanghai, China

1 Introduction

Graph layout technique is the cornerstone for graph visualization, which has wide applications in various fields such as social networks, bioinformatics and chemical engineering. Node-link diagram is most generally used in past decades, through which the topological structure of the graph can be easily comprehended. Early studies attempted to seek a satisfying graph layout through a physical system (Fruchterman and Reingold 1991; Kamada and Kawai 1989). Nowadays, diverse approaches are developed and provide us with more choices on the layouts with different characteristics (Xue et al. 2023; Gibson et al. 2013; Tamassia 2013).

Deep learning is a technique with very enormous potential that is used in the field of computer vision and natural language processing far and wide (LeCun et al. 2015). This technique has also begun to be applied to the computation of graph data in the last decade known as graph neural networks (GNNs). Several popular frameworks such as graph convolutional neural networks (GCNs) (Wu et al. 2021) and graph-LSTM (Liang et al. 2016) are adapted from the combination of graph topology and conceptions of classical CNN and LSTM. More recently, the possibility of graph drawing with the deep learning technique has been successfully verified (Wang et al. 2020) and since then an increasing number of new methods are found to improve the layout quality or help users to generate layout in their preference (Wang et al. 2021; Giovannangeli et al. 2022; Kwon and Ma 2020).

According to the model training strategies, these methods can be divided into two categories: imitation-based and aesthetic-metric-based methods. The imitation-based method like DeepDrawing (Wang et al. 2020) trains models with a large number of graph layouts generated by the existing layout algorithms. The positions of nodes in a graph are regarded as the ground-truth label and the loss function measures the similarity between the predicted and the real layout. The aesthetic-metric-based method such as DeepGD (Wang et al. 2021) belongs to a type of unsupervised training strategy. It employs the aesthetic metrics of graph layout to produce the loss function. The loss function can be designed with a variety of aesthetic metrics, but requires that at least one of the aesthetic metrics can be the basic metric, which is able to draw graphs independently. However, in all of previous studies, there are various deficiencies remained to be solved.

1.1 Training availability

To use a deep learning model to draw graph, we need to train a deep learning model firstly. The quality of the dataset is one of the keys for a successful training. For imitation-based training strategies, when utilized on large graphs, it is expensive to generate a ground-truth layout for each graph in the training dataset and the "correctness" of the layout is too complicated to be assessed, due to the affine transformations and node displacing.

1.2 Scalability

The size of a graph may vary from hundreds to thousands. Therefore, the scalability is significant for a well-trained model. However, there are two main issues that restrict the scalability of model. One is the upper limitation of the spectral-based GNNs (Wu et al. 2021), since this kind of models requires a fixed input size. Another is the heavy architecture, owing to its low computing efficiency.

1.3 Customizability

The fondness of different graph layouts is variant from user to user, which is usually driven by the application scenario, task objective and subjectivity. Although the aesthetic-metric-based training strategies show better availability and scalability on large graphs, stress and t-SNE (Van der Maaten and Hinton 2008) are the only basic metrics implemented currently, of which the customizability is too low to provide users with more options.

In this paper, we conclude a framework that can construct basic metrics from most of the existing graph layout algorithms and design a novel deep-learning-based graph layout approach DeepFD with a generally used layout algorithm, force-directed algorithm, as the basic metric of the loss function. Our model adopts a light-weight "encoder-decoder" structure, graph-LSTM as the encoder and MLP as the decoder, to reduce

the time cost. Additionally, we generate a graph dataset with more than 4000 graphs whose number of nodes are between 500 and 2000, which is quite abundant for model training.

Our approach is compared with two classical layout algorithms and two deep learning models with different structure or loss function to perform both qualitative and quantitative evaluations. In summary, the main contributions of this paper can be concluded as follows:

- A framework to establish basic metrics for unsupervised deep learning graph drawing, which is available for most of the existing layout techniques.
- A deep learning graph drawing with force-directed loss, which can generate a good look layout for large graph with thousands of nodes within a few seconds.
- A comprehensive experimental study to show the efficiency and layout quality of our proposed approach.

2 Related work

This section reviews the related work of this paper, which includes two parts: graph drawing and deep learning techniques on graph drawing.

2.1 Graph drawing

Graph layout has a profound research history, and its earliest research can be traced back to the barycentric method proposed by Tutte (1963). Nowadays, diverse approaches are developed based on different principles such as force based (Fruchterman and Reingold 1991), energy based (Kamada and Kawai 1989), and dimension-reduction based (Van der Maaten and Hinton 2008).

For large graphs, the recent related work mainly focuses on the improvements of the visualizing quality and reductions of time cost. Most of the layout quality optimizations of large graph relies on the visual mapping techniques, such as edge bundling (Zhou et al. 2013) which simplify the links between nodes to expose the graph backbone, and visual metaphor (Dunne and Shneiderman 2013) that encodes the elements in a graph into specific symbols depending on the local structure. As for time cost, the majorization can be in multifarious ways. To reduce the time complexities of layout methods is the most direct way. Barnes and Hut (1986) predigests the computations of repulsions with quadtrees, while decreasing with a node sampling strategy is also adopted (Meidiana et al. 2020). Several studies meliorate the iteration process from the aspect of the optimizer (Xue et al. 2023) and step length (Egorov and Bezgodov 2015). Others refer to the computational architectures such as distributions (Arleo et al. 2019) and GPU (Zellmann et al. 2020). Moreover, using a graph simplifying techniques such as graph sampling (Zhao et al. 2021) or graph aggregation (Ohsaka et al. 2017) can optimize both quality and time cost from the perspective of data size.

2.2 Deep learning techniques on graph drawing

Deep learning graph layout techniques belong to computational improvements according to the taxonomy from Gibson et al. (2013). However, to our best known, there are only five works that apply deep learning techniques to graph drawing directly, while more studies focus on the layout evaluations (Haleem et al. 2019) and explorations (Kwon and Ma 2020).

DeepDrawing (Wang et al. 2020) is recognized as the first deep learning graph drawing techniques. Although it takes the imitation strategy for model training, their work presents a powerful potential on the deep-learning-based graph layout methods. However, this training strategy is not suitable for large graphs. DeepGD (Wang et al. 2021) is the first unsupervised learning techniques, which takes the stress model as the basic metric and explore the permutations among different aesthetic with two adaptive training strategies. It is also tested on large graphs with more than three thousand nodes, but only stress is applicated in this work. GND (Tiezzi et al. 2022) introduced a conception named neural aesthete, that is, training a model with a trained model. Although the trained model is practiced to learn the aesthetic from certain graph layouts, stress is utilized in this work, while it is also the only basic metric of the loss function. $(DNN)^2$ (Giovannangeli et al. 2021) and its extended version DL4GD (Giovannangeli et al. 2022) adapt the ResNet50 (Wu et al. 2019) to achieve convolutions on graphs. Their loss function is designed with a combination of stress and KL divergence, which is an adaptation from t-SNE layout techniques. The most

fatal disability of these inventions is that they both use spectral-based GNNs, hence the input size is limited to a certain number.

3 Framework: basic metrics construction

A graph $G = (V,\ E)$ consists of a node set $V = \{v_1,\ v_2,\ \ldots,\ v_n\}$ and an edge set $E = \{e_{uv} \mid u,\ v \in V\}$. Although there are multiple types of graphs due to their direction, connectivity and weight, we generally simplify the graphs into a non-directed unweight graph and only the largest connected component will be considered. The goal of a graph drawing task is to achieve a position $\mathbf{X}_u$ for each node u in the graph, while reflect the topological structure of the whole graph, while reflect the topological structure of the whole graph. We only discuss the layout in a plane space, where $\mathbf{X}_u \in \mathbb{R}^2$.

3.1 Classical graph layout techniques

Before the introduction of our proposed framework, we briefly review two of the most commonly used graph layout techniques known as force-directed algorithm and stress model.

3.1.1 Force-directed algorithm

The first force-directed algorithm (FDA) was proposed (Eades 1984), known as the "spring- electrical" model. But the formulations put forward by Fruchterman and Reingold (1991) is more well-known, which is usually called as F-R model. For a node u in a graph, the motion of the node is powered by the repulsions $\mathbf{F}_{ru}$ exerted by other nodes and the attractions $\mathbf{F}_{au}$ generated by neighbors, which are calculated respectively as follows:

$$\mathbf{F}_{au} = \sum_{(u,v)\in E} \frac{\|\mathbf{X}_u - \mathbf{X}_v\|^2}{k} \cdot \mathbf{e}_{u,v} \tag{1}$$

$$\mathbf{F}_{ru} = -\sum_{v\in V, v\neq u} \frac{k^2}{\|\mathbf{X}_u - \mathbf{X}_v\|} \cdot \mathbf{e}_{u,v} \tag{2}$$

where $\mathbf{X}_u$ and $\mathbf{X}_v$ represent the coordinates of nodes u and v in the layout space, $\|\mathbf{X}_u - \mathbf{X}_v\|$ denotes the Euclidean distance between nodes u and u, $\mathbf{e}_{u,v}$ is the unit vector defined as $\mathbf{e}_{u,v} = (\mathbf{X}_u - \mathbf{X}_v)/\|\mathbf{X}_u - \mathbf{X}_v\|$ and k is the constraints of layout area.

3.1.2 Stress model

The stress model draws graph based on the potential energy. Each pair of nodes are regarded to be connected by a spring, while its elastic coefficient and natural length depends on the graph-theoretical distance. The general representation of stress model is defined by Kamada and Kawai (K-K) K(amada and Kawai 1989) as follows:

$$min\ \ E = \sum_{\substack{u,\ v\in V \\ v\neq u}} w_{uv}(\|\mathbf{X}_u - \mathbf{X}_v\| - d_{uv})^2 \tag{3}$$

where E is the energy of the system, d_{uv} denotes the graph-theoretical distance while is also the natural length, and w_{uv} represents the elastic coefficient. Typically, w_{uv} is expressed as $1/{d_{uv}}^{\alpha}$, where $\alpha \in \{0, 1, 2\}$ according to Cohen (1997). The Eq. (3) is equivalent to MDS algorithm when $\alpha = 0$, while $\alpha = 2$ is adopted more usual.

3.2 Basic metrics from existing algorithms

Stress models are adopted more usual in current unsupervised deep-learning-based graph drawing approaches since the energy functions can be minimized through gradient descent methods (Gansner et al.

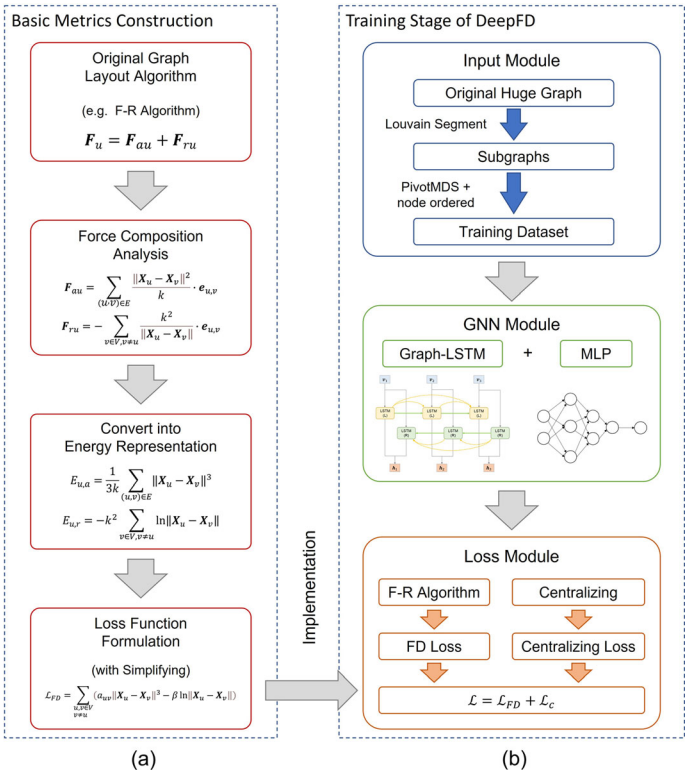


Fig. 1 Workflow of frameworks: **a** basic metrics construction, **b** training stage of DeepFD

2005), which is requisite for the loss function. From the perspective of physical mechanics, energy is the integral of the force over the displacement in a physical system (Noack 2004), which inspires us that every force model is corresponding to an energy representation and the energy function can be deduced by integral. Hence, the force model can be easily converted into an energy model. Given a node u of a graph and its position is $\mathbf{X}_u$, as described in Xue et al. (2023), a kind force of the node u can be unified represented as:

$$\mathbf{F}_{u,t} = \sum_{(u,v)\in\Omega_t} \omega_t \frac{\|\mathbf{X}_u - \mathbf{X}_v\|^{\alpha_t}}{{d_{uv}}^{\beta_t}} \cdot \mathbf{e}_{u,v} \tag{4}$$

where t represents the force kind and Ω_t denotes its effective range. ω_t, α_t and β_t are the coefficients to control the force direction, Euclidean distance and graph-theoretical distance separately. In this formulation, ω_t, α_t, β_t and Ω_t are decided by the definition of the force kind. Since the direction of the force must be along or opposite with $\mathbf{e}_{u,v}$, the potential energy generated by t-kind force of node u can be easily deduced by integral as:

$$E_{u,t} = \begin{cases} \sum\limits_{(u,v)\in\Omega_t} \dfrac{\omega_t}{d_{uv}^{\beta_t}} \cdot \ln\|X_u - X_v\|, & \alpha_t = -1 \\ \sum\limits_{(u,v)\in\Omega_t} \dfrac{\omega_t}{(\alpha_t+1)\cdot d_{uv}^{\beta_t}} \cdot \|X_u - X_v\|^{\alpha_t+1}, & otherwise \end{cases} \tag{5}$$

The total energy of the node u is the summation of $E_{u,t}$ of every kind of force t for each node in the graph. From another perspective of analogy, in a force-based approach, the positions of nodes are moved with the guidance of the summation of all kinds of force in each iteration, which is similar to the role of gradient played in the model training process. Therefore, the join forces can be regarded as the gradient to extrapolate the loss function, which reveals the same expressions as narrated above. On this basis, we implement the force-directed algorithm to establish a basic metric and propose DeepFD as described in Sect. 4. The workflow of this framework is shown as Fig. 1a.

4 DeepFD design

This section we introduce an unsupervised deep learning approach DeepFD to fast generate force-directed layout for large graphs. Our approach is described from the perspectives of model architecture, model input and loss function.

4.1 Model architecture

We construct our model on the "encoder-decoder" architecture, where the encoder generates the hidden representations from the node input features and decoder produces the node positions via the encoded information. We proposed to use a graph-LSTM as the encoder and adopted the classical multi-layer perceptron (MLP) as the decoder. Figure 1b shows its workflow on the training stage.

4.1.1 Encoder: graph-LSTM

Although graph drawing is a graph-level task, it has to generate a node-level output owing that the position of every node is bound to be figured out. Therefore, our model should be able to efficiently aggregate the node features of the whole graph. Spectral-based GNNs are not applied to this work since they have poor generalization capability on the graph size. GCNs can welly gather the node features from nodes neighborhoods, but they require deep layers to enhance its feature extraction from global range, which is very time-consuming (Wu et al. 2021). Additionally, the outputs will not change with the same inputs, which means that if several nodes are embedded into the same representation, the layouts from GCNs will show significant overlaps.

Considering that graph is a typical sequential data, our model is chosen from RNN-based networks which can better accumulate features from the whole graph with high efficiency and the outputs of the nodes with same embeddings can be various from different time step. Nevertheless, graph is a nonlinear data with complex topology, making it difficult to achieve ideal results through linear models such as LSTM. We propose to use graph-LSTM (Liang et al. 2016), which largely improves the capability of LSTM model on the nonlinear sequence data. This kind of model adds several "links" according to the topological structure of the graph. The transition functions of graph-LSTM are shown as equations from (7)–(13).

$$\mathbf{f}_{t,t-1} = \sigma\left(\mathbf{W}_f\ \mathbf{x}_t + \mathbf{U}_f\mathbf{h}_{t-1} + \mathbf{b}_f\right) \tag{6}$$

$$\mathbf{f}_{t,k} = \sigma\left(\mathbf{W}_f\ \mathbf{x}_t + \widetilde{\mathbf{U}}_f\mathbf{h}_k + \mathbf{b}_f\right),\ v_k \in P(v_t) \tag{7}$$

$$\mathbf{i}_t = \sigma\left(\mathbf{W}_i\ \mathbf{x}_t + \mathbf{U}_i\mathbf{h}_{t-1} + \sum_{v_k \in P(v_t)} \widetilde{\mathbf{U}}_i\mathbf{h}_k + \mathbf{b}_i\right) \tag{8}$$

$$\widetilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_c\ \mathbf{x}_t + \mathbf{U}_c\mathbf{h}_{t-1} + \sum_{v_k \in P(v_t)} \widetilde{\mathbf{U}}_c\mathbf{h}_k + \mathbf{b}_c\right) \tag{9}$$

$$\mathbf{c}_t = \mathbf{f}_{t,t-1} \odot \mathbf{c}_{t-1} + \sum_{v_k \in P(v_t)} \mathbf{f}_{tk} \odot \mathbf{c}_k + \mathbf{i}_t \odot \widetilde{\mathbf{c}}_t \tag{10}$$

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_o\ \mathbf{x}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \sum_{v_k \in P(v_t)} \widetilde{\mathbf{U}}_o\mathbf{h}_k + \mathbf{b}_o\right) \tag{11}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot tanh(\mathbf{c}_t) \tag{12}$$

where v_t, $\mathbf{x}_t$, $\mathbf{h}_t$, $\mathbf{c}_t$ and $\widetilde{\mathbf{c}}_t$ denote the input node, input feature vector, hidden state, cell state and candidate cell state at the current tick t, separately, where $\mathbf{x}_t \in \mathbb{R}^d$ and $\mathbf{h}_t$, $\mathbf{c}_t$, $\widetilde{\mathbf{c}}_t \in \mathbb{R}^h$. $P(v_t)$ defined as $\{v_i | v_i \in \mathcal{N}(v_t),\ i < t\}$ refers to the set of prior nodes linked toward node v_t, where $\mathcal{N}(v_t)$ is the neighborhood of node v_t. $\mathbf{f}_{t,t-1}$, $\mathbf{i}_t$ and $\mathbf{o}_t$ are the forgotten gate, input gate and output gate from the standard LSTM, while $\mathbf{W}$'s and $\mathbf{U}$'s are the network layers for the input and hidden state and $\mathbf{b}$'s are bias vectors. Since the new "links" are added, the hidden and cell states from the former nodes in $P(v_t)$ are taken into the

computation at current tick. $\mathbf{f}_{t,k}$ is the additional forgotten gate to limit the information flowing from cell state at tick k. In the candidate cell state and each gate, new matrices $\widetilde{\mathbf{U}}$'s is used to control and integrate the hidden states from prior nodes. Hence, the ameliorative architecture can reflect the real dependencies between nodes and links, while also distill the node features in global range.

The detailed architecture of the encoder is depicted as in Fig. 2. The bidirectional propagation design allows our model to gain the hidden features from both the former and the latter nodes in the input sequence. More specifically, we provide an example to describes how the states flow in our model. Consider the undirected graph depicted in Fig. 2a: the input node order in the graph is $Q_V = (v_1, v_2, v_3)$. Then, the streams of the data are represented by the arrows in Fig. 2b. The dotted orange arrows signify the new "links" that propagate the message according to the graph topological structure, and the solid green arrows are the links from standard LSTM. The node ordering for the backward is permuted by reversing the input sequence of the forward. The hidden states in both forward and backward propagation are concatenated as the output from the encoder, which belongs to $\mathbb{R}^{2h}$.

4.1.2 Decoder: MLP

Graph layout is a typical complex nonlinear problem. Although graph-LSTM can better aggregate the topological properties of nodes among the global and neighborhood than Bi-LSTM, it is still a single layer network which shows poor effect when facing with complex regression problems. More specifically, graph-LSTM is an "encoder" who gathers the structural features for every node and embeds the nodes into a Hilbert space, while we need to seek out another "decoder" to figure out the planar position of each node from the high-dimensional representation.

Multilayer perceptron (MLP) (Taud and Mas 2018) is an important model structure to enhance the nonlinear tasks handling capability of deep learning model, which is often adopted as the decoder of a model to generate the final results. The more layers in MLP will perform a better partition of the solution space but will also increase the risk of under- or over-fitting. The number of sensory neurons in the hidden layer determines how complex the issue can be solved through the model. Activation function plays a key role in the nonlinear problem settling while it also affects the fitting degree during the model training.

We designed a 4-layered MLP with three hidden layers and an output layer. According to a study from Xie et al. (2021), instead of ReLU, we use CeLU as the activation function for the first three layers to improve the robustness of the proposed model.

4.2 Model input

Finding an appropriate way to order the nodes in graphs is critical when using an RNN-based model. This section expounds our design on the node embedding and the input order.

4.2.1 Node embedding

Graph embedding is the first step when applying a deep learning model to graphs. Through a transformation from nodes to fixed-dimensional representations, each node can be indicated by a vector to be the input of a deep learning model. Random-walk-based methods such as DeepWalk (Perozzi et al. 2014) and node2vec (Grover and Leskovec 2016), and deep-learning-based approaches like SDNE (Wang et al. 2014) can efficiently extract the topological features within a short time. However, these embedding techniques have been proved unavailable for multiple graphs since they cannot generate a unified representations among different graphs (Heimann and Koutra 2017). Factor-decomposition-based methods such as Laplace eigenvalue decomposition (Belkin and Niyogi 2003) can provide stable embedding for multiple graphs, nonetheless, the time cost of this kind of methods are usually expensive, which is fatal for the computation on large graphs. Applying a fixed-length adjacency vector will leads to a sparse representation contenting little structural information of a graph.

From our prior surveys, PivotMDS (Brandes and Pich 2007) is a dimension reduction technique with only a linear time complexity and is frequently used in previous graph layout researches to initialize node positions. More recently, this technique has been verified to be an effective graph embedding approach in the related researches (Wang et al. 2021; Giovannangeli et al. 2021). Hence, we utilize PivotMDS to produce embedding representations of high-dimensional nodes in this work.

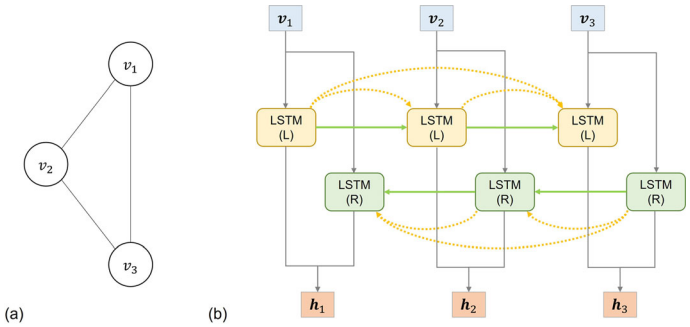


Fig. 2 An example of data flows in graph-LSTM. **a** A graph with three nodes sequenced as $Q_V = (v_1,\ v_2,\ v_3)$. **b** The data flow in our proposed model where LSTM(L) is the forward propagation and LSTM(R) is the backward

4.2.2 Node input order

The graph-LSTM-based models require the input data to be in the form of a sequence. If the nodes are ordered randomly, the number of all possible node permutations for a connected graph is $n!$, which results in an excessive search space. We employ a way introduced by You et al. (2018) to organize the node sequences using breadth-first-search (BFS) to substantially reduce the variety of the possible node ordering. However, the permutation of nodes in a specific level of BFS sorting can be complicated, and the start node of a BFS can be any.

Selecting the highest-degree node as the start may lead to an unpredictable result. For example, there are two graphs shown in Fig. 3. The degrees of the nodes in the center are both 6, but their topological structures are quite different. The graph in Fig. 3(a) shows a star topology, where the central node plays a crucial role in the connectivity of the graph; As for Fig. 3b, any pair of nodes is connected, and thus all nodes are equivalent. However, these two graphs will be ordered in a same permutation, which might confuse the recognition of model.

In contrast, a node with the lowest degree can significantly minimize the impact of the diverse topology since this kind of nodes can only be on the boundary or a bridge of a graph. We choose the lowest-degree node in a graph to start BFS and order nodes with the rank of the degree in each level. If there are several nodes that all have the minimal degree, one of the nodes will be randomly chosen to start BFS.

4.3 Loss function

For large graphs, it is expensive to generate a ground-truth layout for each graph in the training dataset and the similarity of layouts between the generated and ground truth is too complicated to be measured through a loss function. Compared to imitation strategy, designing a loss function that can train model in an unsupervised way is more desirable. We take force-directed model as the prototype to formulate the FD loss function through the framework developed in Sect. 3.2.

4.3.1 FD loss

The definitions of attraction and repulsion of force-directed algorithm are provided as Eqs. (1) and (2). Hence, for a node u in a graph, the energies of attraction and repulsion for the node u at the position $\mathbf{X}_u$ are $E_{u,a}$ and $E_{u,r}$, and the FD loss of node u is defined as:

$$\begin{aligned}\mathscr{L}_{FD}(u) &= E_{u,a} + E_{u,r} \\ &= \sum_{v \in V, v \neq u} \left(\frac{a_{uv} \|\mathbf{X}_u - \mathbf{X}_v\|^3}{3k} - k^2 \ln \|\mathbf{X}_u - \mathbf{X}_v\| \right)\end{aligned} \tag{13}$$

where a_{uv} represents the value of u-th row and v-th col in the adjacency matrix. Since the extreme point would not change when the function is multiplied with a nonzero number, to simplify the coefficients, the loss function is multiplied with $3k$ and a new coefficient $\beta = 3k^3$ is implemented. Sum the $\mathscr{L}_{FR}(u)$ for each node u in the graph to achieve the FD loss function as:

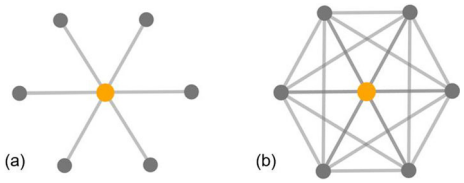


Fig. 3 Two example graphs consist of seven nodes with different structures: **a** star, **b** full connect. The degrees of nodes in the center of each graph are both six

$$\mathscr{L}_{FD} = \sum_{\substack{u,\ v \in V \\ v \neq u}} \left(a_{uv} \|\mathbf{X}_u - \mathbf{X}_v\|^3 - \beta \ln \|\mathbf{X}_u - \mathbf{X}_v\| \right) \tag{14}$$

In previous studies, the coefficient k was adopted to limit the layout in a fixed area, which can be replaced with a series of operations such as translation, rotation and scaling. Thus β can be any positive value, but is recommended to set $\beta = \sqrt{n}$ to keep the value of $\mathscr{L}_{FD}$ in a computable range.

4.3.2 Centralizing loss

Centralizing force is mostly employed as a restraint to keep the outline of layout in a regular shape. For example, if there is a graph with a long chain as shown in Fig. 4, the layout generated by force-directed model would display with a long and narrow shape as Fig. 4a, which is not desirable and the expected layout should look like Fig 4b. The formulation of centralizing force $\mathbf{F}_{cu}$ is:

$$\mathbf{F}_{cu} = \mu(\mathbf{X}_u - \mathbf{X}_O) \tag{15}$$

where $\mathbf{X}_O$ denotes the center of the layout and μ is a coefficient. $\mathbf{X}_O$ is used to be calculated as $\mathbf{X}_O = \sum_{v \in V} \mathbf{X}_v / n$. However, this expression does not take the weight of nodes into consideration, which may distort the layout of the majority in the graph. We utilize degrees to weigh the nodes to compute the center position as follows:

$$\mathbf{X}_O = \frac{\sum_{v \in V} deg(v) \cdot \mathbf{X}_v}{\sum_{v \in V} deg(v)} \tag{16}$$

where $deg(v)$ represents the degree of node v. Similarly, the centralizing loss can be derived from integral by regarding the centralizing force as gradient, which is formulated as follows:

$$\mathscr{L}_c(\mathbf{X}_u) = \frac{1}{2} \mu \|\mathbf{X}_u - \mathbf{X}_O\|^2 \tag{17}$$

Let $\mu = \mu/2$ to simplify the constant and sum the loss of all nodes, the centralizing loss is developed as follows:

$$\mathscr{L}_c = \mu \sum_{u \in V} \|\mathbf{X}_u - \mathbf{X}_O\|^2 \tag{18}$$

where μ is recommended to be $1/\sqrt{n}$ due to the quadratic linear relationship between the layout area and the node count (Fruchterman and Reingold 1991).

4.3.3 Combination strategy

The combination of multiple aesthetic metrics can be simply weighted summed as:

$$\mathscr{L} = \mathscr{L}_0 + \sum_{t=1} \lambda_t \mathscr{L}_t \tag{19}$$

Where $\mathscr{L}_0$ is the basic metric and the λ_t's are the coefficients that can be self-adaptive or deduced from previous studies. Owing that there is exactly a factor μ in the $\mathscr{L}_c$, the loss functions can be easily combined as follow:

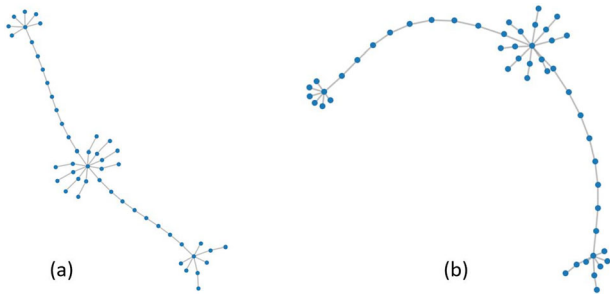


Fig. 4 Layouts of a graph with a long chain: **a** without, and **b** with the centralizing force

$$\mathscr{L} = \mathscr{L}_{FD} + \mathscr{L}_c \tag{20}$$

5 Evaluation

We conduct a set of qualitative and quantitative experiments to evaluate DeepFD. This section detailly introduces experiment settings and exhibits the evaluation results.

5.1 Experiment design

5.1.1 Dataset

Training dataset Since most of the existing public graph datasets consist of large numbers of small graphs like Rome[1] or a huge single graph as com-YouTube[2], it is laborious to gather a large quantity of graphs with hundreds or thousands of nodes. To generate a graph dataset, earlier works prefer to employ graph generative methods to obtain a large amount of graph. On the contrary, we adopt a "split" strategy and design a multi-level segment approach to divide huge graphs with millions of nodes into segments and preserve the segmented subgraphs that are eligible for the training dataset. These subgraphs often have diverse structural features that can strengthen the robustness of the model. We use Louvain (Traag et al. 2019) algorithm to partition subgraphs from com-YouTube[2], LiveJournal[2], Pokec[3] and Orkut[2] datasets. The subgraphs with nodes count between 500 to 2000 are reserved and the subgraphs with more than 2000 nodes are divided in an additional partition. Finally, we gain a dataset with 4453 large graphs. The graphs are randomly split into three parts among which there are not any intersection: 4272 for training, 128 for testing and 53 for validation. We generate a ground-truth layout for each data by F-R algorithm to perform the training of DeepDrawing, which adopts a imitation strategy. This process is turned out to be time-consuming that it takes us about 33 hours to finish.

Ground-truth dataset We utilize 15 public ground-truth large graphs[3] from prior studies to access DeepFD. The scale of the graphs ranges from 744 to 6474 and the edge number ranges from 2112 to 88234. All of the graphs are used to evaluate quantitatively and 4 of them to perform case studies.

5.1.2 Baseline methods

The baseline methods consist of traditional graph layout algorithms and deep-learning-based models. For the traditional algorithms, we utilize two well-known and widely used models, F-R model (Fruchterman and Reingold) and K-K model (Kamada and Kawai 1989), to access the efficiency and layout quality of DeepFD. For the deep-learning-based approaches, we trained several models to perform ablation experiments. A graph-LSTM-based models without MLP layers is trained with FD loss to evaluate the necessity of the MLP layer in the graph layout task. We also implement DeepDrawing (Wang et al. 2020) to show the disadvantage of the "imitation" strategy and generate layout with F-R algorithm for each graph of the

[1] www.graphdrawing.org/data.html.

[2] socialnetworks.mpi-sws.org/data-imc2007.html.

[3] snap.stanford.edu/data/soc-Pokec.html.

training dataset. Furthermore, a model with the same architecture as DeepFD is trained with the stress loss function to evaluate the layout qualities of different loss functions.

5.1.3 Implementation and training configuration

We carry out the training and evaluation process on Linux system with 40 Intel@ Xeon(R) CPU processors (E5-2630 v4, 2.20GHz) and a GeForce RTX 3090 GPU. The deep learning models are implemented with PyTorch and PyG Library. F-R and K-K models are implemented with the Python NetworkX package.

The configurations for the graph-LSTM and MLP layers are listed in Table 1. We train the models with an Adam optimizer. For each model, the learning rate and batch size are set to 0.003 and 16. The size of the input vectors of nodes are set as 128. Each model is trained on the training dataset for 100 epochs.

5.2 Quantitative evaluation

5.2.1 Training loss

Loss lines can intuitively reflect the convergence efficiency of model and loss function during the model training. We record the FD loss of the graph-LSTM-based models with and without MLP, and the DeepDrawing on the testing dataset for 100 epochs.

As provided in Fig. 5a, all of the loss lines show significant downward trends, which means that the models are trained effectively with the guidance of the loss functions. However, the loss lines of two models trained with FD loss are significantly lower than DeepDrawing. Additionally, the graph-LSTM + MLP model represents lower and smoother than the graph-LSTM model. The results indicate that the aesthetic-metric-based strategy shows evident advantage toward the imitation-based and decoder like MLP is essential to enhance the performance and robustness.

5.2.2 Time analysis

Our proposed approach is compared with K-K algorithm and F-R algorithm to verify the significant advantages of DeepFD on computational speed. The time costs of deep-learning model are tested both on CPU and GPU and the batch size is set to 1. The iteration number of F-R model is limit to 200 steps since the layout is nearly settled after 200 steps. This experiment is conducted on all of the 15 ground-truth large graphs. For each graph, each candidate is run repeatedly for 10 times and take the mean value of the runtime as the real time cost.

Table 2 presents the time cost of each layout approach on 15 large graphs. The table is ordered by the number of nodes in the graphs. The result shows that on both CPU and GPU modes, our approach has evident advantages compared with the traditional layout techniques. On CPU mode, the time cost of our proposed model is only 7.43% of F-R and 2.06% of K-K on average, while it is 9.57% of F-R and 2.65% of K-K on GPU mode. The advantages also behave a growing tendency with the node count increasing. The layout speed of our method on CPU is about 4 times faster than F-R and about 7 times faster than K-K on Facebook1912 dataset, while these two numbers go to 21 and 99 on AS-733 dataset. The model without MLP shows a bit faster due to the lesser parameters. Nevertheless, the time cost of graph-LSTM-based models on CPU shows a bit lower than on GPU, which seems to be unusual. Since the batch size is set to 1, the migration of data between RAM and VRAM can be costly and the strength of GPU computation is not able to be reflected within a small input.

Table 1 The parameter configurations of our model

Structure	Parameter name	Value
Graph-LSTM	Input size	128
	Hidden size	256
	Layers	1
	Direction	Bidirectional
MLP	Hidden size	256
	Hidden layers	3
	Activation function	CeLU
Output layer	Output size	2

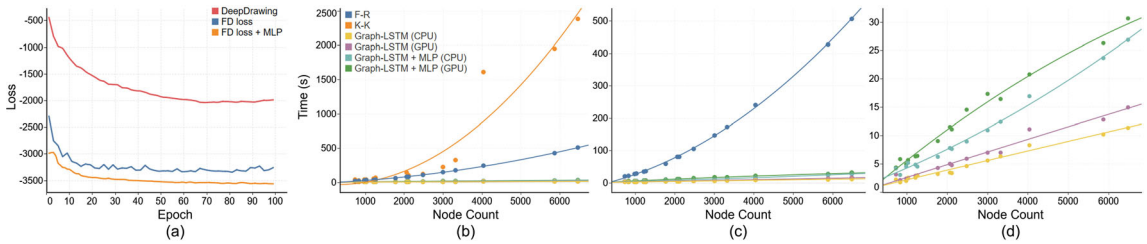


Fig. 5 **a** FD loss value of the graph-LSTM-based models with MLP (orange), without MLP (blue) and DeepDrawing(red) during training process. **b**, **c**, **d** Plots of runtime versus node number for four candidates. **b** Full view. **c** Without K-K. **d** Deep learning models only

Table 2 Running time of the traditional layout techniques (F-R and K-K) and the graph-LSTM-based models (on CPU and GPU) on fifteen large graphs

Dataset	Node	Edge	Traditional		Graph-LSTM-based models			
			F-R	K-K	Without MLP		With MLP (Ours)	
					CPU	GPU	CPU	GPU
Facebook1912	744	30023	19.87	35.62	2.19	3.08	4.15	4.31
Cpan	839	2112	20.41	22.83	1.70	2.07	3.04	5.83
email-Eu	986	16687	26.29	39.59	1.97	2.32	4.47	4.90
Facebook107	1034	26749	28.77	39.06	2.51	2.39	5.17	5.65
polblogs	1222	16717	34.72	53.96	2.48	2.83	4.81	6.28
Eurosis	1272	6454	35.47	64.41	2.95	2.96	4.48	6.37
DS_AT	1777	4323	57.81	61.79	3.17	4.23	6.18	9.00
OS	2068	8654	80.63	137.31	3.39	4.95	7.81	11.46
CiteSeer	2120	3731	80.23	99.37	3.36	4.74	7.65	11.08
cora	2485	5069	104.86	122.38	4.54	5.88	8.89	14.54
Programming	3007	10479	145.95	222.41	5.58	6.97	10.88	17.34
AI_ML	3323	10603	172.95	322.50	6.32	6.95	12.41	16.42
Facebook_Full	4039	88234	240.41	1610.62	8.25	11.08	16.87	20.75
Bitcoinotc	5875	21489	427.63	1945.97	10.14	12.80	23.65	26.30
AS-733	6474	12572	507.01	2386.02	11.34	14.91	26.81	30.61

To better learn the time complexities of these solvers, we employ a scatter plot and utilize quadratic regression lines to reveal the relationship between runtime and number of nodes for the approaches in Table 2. It is obvious that the K-K algorithm presents an ultralinear time complexity in Fig. 5b and so does F-R algorithm as is shown in Fig. 5c. As for deep learning methods, see Fig. 5d, the time complexity is approximate to linear on CPU and to sublinear on GPU, which indicates that our proposed model is capable for large graph layout and on larger graphs, the performance on GPU will be better.

5.3 Qualitative evaluation

Our qualitative evaluation is performed on four ground-truth large graphs as in Fig. 6 to reveal the learning capability of different strategies and model architectures. F-R and K-K algorithms are utilized to show the expected layouts and for each deep-learning-based method, the model saved in the last epoch is employed for comparisons.

Cpan (first row) and Eurosis (second row) datasets are large graphs with copious edge features and clear structure. For traditional layout techniques, F-R algorithm vividly depicts the outline of the edge features, while K-K algorithm reflects the structure more specifically with the hierarchic clusters like an “onion,” For deep-learning-based methods, however, all of the trained models draw irregular shapes on both of two graphs. The model trained by stress loss draws a ball of messes, where neither edge nor structural features are reflected obviously. The layouts generated by graph-LSTM and DeepDrawing shows chaos nodes layouts and cross edges. Although the layouts drawn by DeepFD do not look so pretty as the drawn by traditional techniques, they are still able to clearly express the outlines and backbones of the graphs, which denotes that our proposed method can better learn the structural features from global range and draw graph layouts in a more proper way.

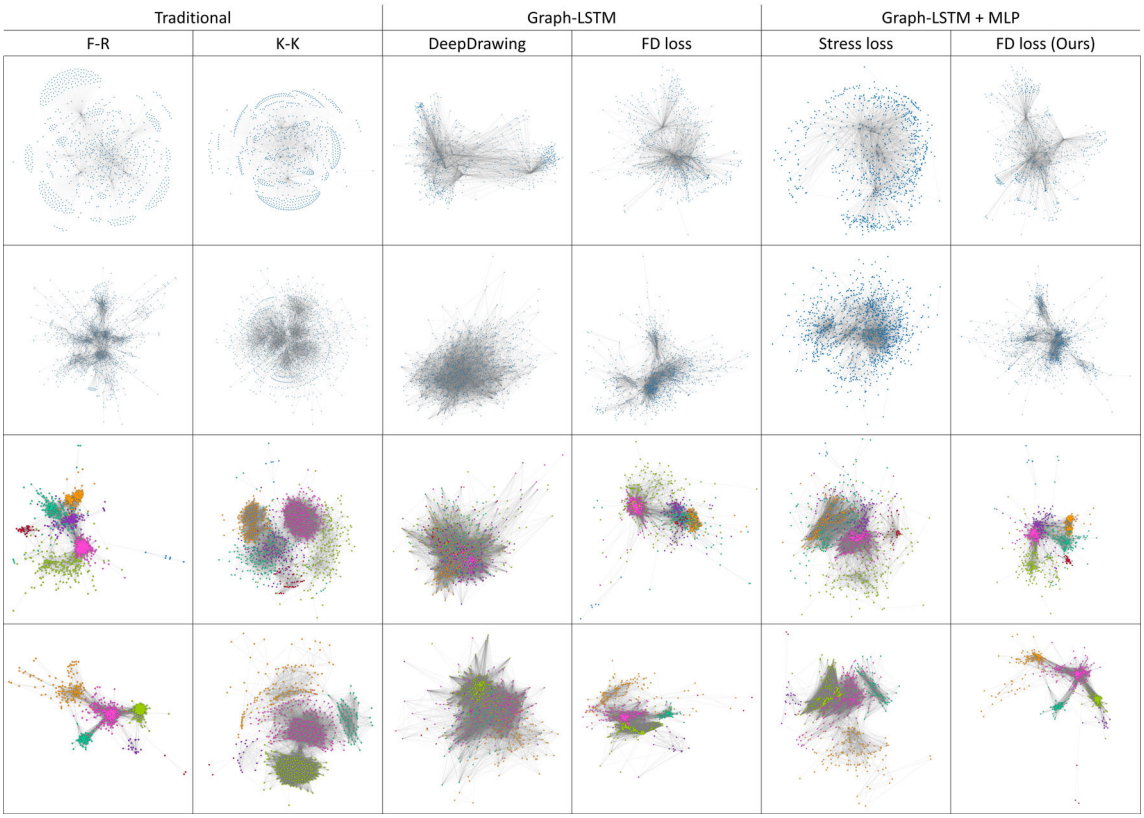


Fig. 6 Qualitative evaluation of DeepFD on four graphs (top down: Cpan, Eurosis, Facebook107 and Facebook1912) in comparison to traditional (F-R and K-K) and deep-learning-based (DeepDrawing, graph-LSTM + FD loss and graph-LSTM + MLP + stress loss) baseline methods

The last two rows of Fig. 6 consist of Facebook107 (third row) and Facebook1912 (last row) datasets, which are large graphs with ground-truth communities. The nodes of different communities are labeled with various colors. K-K algorithm reveals the actual graph-theoretical distance between nodes, but the boundaries between clusters are ambiguous and several clusters overlap in the layouts. These issues are observed in the layouts drawn by the model with stress loss either. F-R algorithm, DeepDrawing and the models trained with FD loss alleviate the above problems and figuratively describe the edge features. However, compared to DeepFD, other deep-learning-based layout methods generate clusters with more distorted boundaries that are much fuzzier at different levels. These outcomes show a evident strength that our developed DeepFD can better learn the community structures and exactly reflect the clustered nodes in the graph layouts.

On the whole, DeepFD exhibits a sound performance in the layout task of large graphs. Above results also indicates that our proposed framework is potential for improving layout quality of deep learning approaches on large graphs.

6 Discussions

The results above evaluations indicate that our proposed DeepFD can learn to efficiently generate force-directed graphs layouts in an unsupervised manner. There are still some limitations and areas for improvement in this study.

6.1 Permutations of forces

Although we introduce a framework that can provide flexible loss function designs for graph layout and we bring up an implementation with F-R algorithm, how would the permutations of different force types influence the training model is an issue that deserves to conduct more in-depth research in the future.

6.2 Model architecture and parameters

We refer to the commonly used "encoder-decoder" architecture in graph deep learning and adopt a lightweight graph-LSTM-based model. The parameter settings of the model are based on previous research experience, but how these parameters affect our model needs further experiments, which is also a key issue that needs to be addressed in terms of model interpretability. In addition, whether there is a better encoder or decoder architecture to replace the current model can be further explored in the future.

6.3 Quality of training dataset

The performance of deep learning models largely depends on the quality of the training dataset. The number and proportion of different types of samples covered by the dataset will affect the generalization ability of model when encountering unseen data. Although we segmented out a large number of large graphs using the Louvain algorithm as training datasets, the quality and the distribution of the dataset are not analyzed in this work. It means that we need to further assess the training dataset and conduct detailed statistics and evaluation on the composition of the data, and that is also a time-consuming process.

7 Conclusion

We design a force-based framework for the construction of basic metrics, which can transform all force-based graph layout algorithms into differentiable energy function forms. This kind of physical system represented by an energy form can be minimized through gradient descent, hence it can also be used to guide the training of graph layout in deep learning models. Furthermore, we build a loss function based on F-R algorithm and develop a graph-LSTM-based layout technique DeepFD. We use the multilevel Louvain algorithm to partition huge graphs with millions of nodes and preserve subgraphs within a certain scale to provide a plentiful dataset for model training. Finally, the results of quantitative and qualitative evaluations also demonstrate that DeepFD exhibits linear time complexity on the CPU and can generate high-quality graph layouts in few seconds.

In the feature, we would like to expand our works from the following aspects. First, we decide to explore more deep learning architectures and figure out several models that suitable for large graph computations, which can generate a desirable result within few seconds. Second, we plan to work on the combinations of different force formulations and more aesthetic metrics such as node occlusion, edge length and average angle. We also plan to construct a self-adaptive multi-objective training strategy that can organize loss functions in a proper way.

Acknowledgements This work was supported by National Natural Science Foundation of China (No. 61872432).

References

Arleo A, Didimo W, Liotta G, Montecchiani F (2019) A distributed multilevel force-directed algorithm. IEEE Trans Parallel Distrib Syst 30(4):754–765. https://doi.org/10.1109/TPDS.2018.2869805

Barnes J, Hut P (1986) A hierarchical o (n log n) force-calculation algorithm. Nature 324(6096):446–449

Belkin M, Niyogi P (2003) Laplacian eigenmaps for dimensionality reduction and data representation. Neural Comput 15(6):1373–1396. https://doi.org/10.1162/089976603321780317

Brandes U, Pich C (2007) Eigensolver methods for progressive multidimensional scaling of large data. In: Kaufmann M, Wagner D (eds) Graph drawing. Springer, Berlin, pp 42–53

Cohen JD (1997) Drawing graphs to convey proximity: an incremental arrangement method. ACM Trans Comput Hum Interact 4(3):197–229. https://doi.org/10.1145/264645.264657

Dunne C, Shneiderman B (2013) Motif simplification: Improving network visualization readability with fan, connector, and clique glyphs. In: Proceedings of the SIGCHI conference on human factors in computing systems, CHI '13. Association for Computing Machinery, New York, pp 3247–3256. https://doi.org/10.1145/2470654.2466444

Eades P (1984) A heuristic for graph drawing. In: Proceedings of the 13th Manitoba conference on numerical mathematics and computing, vol. 24. Utilitas Mathematica, University of Manitoba, Winnipeg
Egorov D, Bezgodov A (2015) Improved force-directed method of graph layout generation with adaptive step length. Procedia Comput Sci 66:689–696
Fruchterman TMJ, Reingold EM (1991) Graph drawing by force-directed placement. Softw Pract Exp 21(11):1129–1164. https://doi.org/10.1002/spe.4380211102
Gansner ER, Koren Y, North S (2005) Graph drawing by stress majorization. In: Pach J (ed) Graph drawing. Springer, Berlin, pp 239–250
Gibson H, Faith J, Vickers P (2013) A survey of two-dimensional graph layout techniques for information visualisation. Inf Vis 12(3–4):324–357. https://doi.org/10.1177/1473871612455749
Giovannangeli L, Lalanne F, Auber D, Giot R, Bourqui R (2021) Deep neural network for drawing networks, $(DNN)^2$. Graph drawing and network visualization. Springer International Publishing, Cham, pp 375–390
Giovannangeli L, Lalanne F, Auber D, Giot R, Bourqui R (2022) Toward efficient deep learning for graph drawing (DL4GD). IEEE Trans Vis Comput Graph. https://doi.org/10.1109/TVCG.2022.3222186
Grover A, Leskovec J (2016) Node2vec: scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, KDD '16. Association for Computing Machinery, New York, pp 855–864. https://doi.org/10.1145/2939672.2939754
Haleem H, Wang Y, Puri A, Wadhwa S, Qu H (2019) Evaluating the readability of force directed graph layouts: a deep learning approach. IEEE Comput Graph Appl 39(4):40–53. https://doi.org/10.1109/MCG.2018.2881501
Heimann M, Koutra D (2017) On generalizing neural node embedding methods to multi-network problems. In: KDD MLG workshop
Kamada T, Kawai S et al (1989) An algorithm for drawing general undirected graphs. Inf Process Lett 31(1):7–15
Kwon O-H, Ma K-L (2020) A deep generative model for graph layout. IEEE Trans Visual Comput Graph 26(1):665–675. https://doi.org/10.1109/TVCG.2019.2934396
LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521(7553):436–444
Liang X, Shen X, Feng J, Lin L, Yan S (2016) Semantic object parsing with graph lstm. In: Leibe B, Matas J, Sebe N, Welling M (eds) Computer vision—ECCV 2016. Springer International Publishing, Cham, pp 125–143
Meidiana A, Hong S-H, Torkel M, Cai S, Eades P (2020) Sublinear time force computation for big complex network visualization. Comput Graph Forum 39(3):579–591. https://doi.org/10.1111/cgf.14003
Noack A (2004) An energy model for visual graph clustering. In: Liotta G (ed) Graph drawing. Springer, Berlin, pp 425–436
Ohsaka N, Sonobe T, Fujita S, Kawarabayashi K-i (2017) Coarsening massive influence networks for scalable diffusion analysis. In: Proceedings of the 2017 ACM international conference on management of data, SIGMOD '17. Association for Computing Machinery, New York, pp 635–650. https://doi.org/10.1145/3035918.3064045
Perozzi B, Al-Rfou R, Skiena S (2014) Deepwalk: online learning of social representations. In: Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data mining, KDD '14. Association for Computing Machinery, New York, pp 701–710. https://doi.org/10.1145/2623330.2623732
Tamassia R (2013) Handbook of graph drawing and visualization. CRC Press, Boca Raton
Taud H, Mas J (2018) Multilayer perceptron (MLP). Springer International Publishing, Cham, pp 451–455
Tiezzi M, Ciravegna G, Gori M (2022) Graph neural networks for graph drawing. IEEE Trans Neural Netw Learn Syst. https://doi.org/10.1109/TNNLS.2022.3184967
Traag VA, Waltman L, Van Eck NJ (2019) From Louvain to Leiden: guaranteeing well-connected communities. Sci Rep 9(1):5233
Tutte WT (1963) How to draw a graph. Proc Lond Math Soc 3(1):743–767
Van der Maaten L, Hinton G (2008) Visualizing data using t-SNE. J Mach Learn Res 9:2579
Wang X, Yen K, Hu Y, Shen H-W (2021) DeepGD: a deep learning framework for graph drawing using GNN. IEEE Comput Graph Appl 41(5):32–44. https://doi.org/10.1109/MCG.2021.3093908
Wang Y, Jin Z, Wang Q, Cui W, Ma T, Qu H (2020) Deepdrawing: a deep learning approach to graph drawing. IEEE Trans Visual Comput Graph 26(1):676–686. https://doi.org/10.1109/TVCG.2019.2934798
Wang Y, Yao H, Zhao S (2016) Auto-encoder based dimensionality reduction. Neurocomputing 184:232–242. https://doi.org/10.1016/j.neucom.2015.08.104
Wu Z, Pan S, Chen F, Long G, Zhang C, Yu PS (2021) A comprehensive survey on graph neural networks. IEEE Trans Neural Netw Learn Syst 32(1):4–24. https://doi.org/10.1109/TNNLS.2020.2978386
Wu Z, Shen C, van den Hengel A (2019) Wider or deeper: revisiting the resnet model for visual recognition. Pattern Recogn 90:119–133. https://doi.org/10.1016/j.patcog.2019.01.006
Xie C, Tan M, Gong B, Yuille A, Le QV (2021) Smooth adversarial training
Xue M, Wang Z, Zhong F, Wang Y, Xu M, Deussen O, Wang Y (2023) Taurus: towards a unified force representation and universal solver for graph layout. IEEE Trans Visual Comput Graph 29(1):886–895. https://doi.org/10.1109/TVCG.2022.3209371
You J, Ying R, Ren X, Hamilton W, Leskovec J (2018) GraphRNN: generating realistic graphs with deep auto-regressive models. In: Dy J, Krause A (eds) Proceedings of the 35th international conference on machine learning, vol 80. PMLR, 10–15 July, pp 5708–5717
Zellmann S, Weier M, Wald I (2020) Accelerating force-directed graph drawing with rt cores. In: 2020 IEEE visualization conference (VIS), pp 96–100. https://doi.org/10.1109/VIS47514.2020.00026
Zhao Y, Jiang H, Chen Q, Qin Y, Xie H, Wu Y, Liu S, Zhou Z, Xia J, Zhou F (2021) Preserving minority structures in graph sampling. IEEE Trans Visual Comput Graph 27(2):1698–1708. https://doi.org/10.1109/TVCG.2020.3030428
Zhou H, Xu P, Yuan X, Qu H (2013) Edge bundling in information visualization. Tsinghua Sci Technol 18(2):145–156. https://doi.org/10.1109/TST.2013.6509098

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.